SIMATS ONLINE PROGRAMMING LAB

PYTHON PROGRAMMING — ASSIGNMENT 4 TECHNICAL REPORT

Student Name	Pughzhvanan D
Register Number	1912260281
Course & Code	CSA0809 — Python Programming (CO4 Coverage)
Repository Format	CSA08-1912260281-Assignment4
Repository URL	`https://github.com/ppughzhvanan-byte/CSA08-1912260281-Assignment4.git`
Total Marks & Bloom's	50 Marks (Tasks 1 to 6) | K3 - Apply, K4 - Analyse, K5 - Evaluate

TABLE OF CONTENTS

1. **Task 1:** File-Processing Design Review: Diagnose and Repair a Poor Script (6 Marks | K4 Analyse)
2. **Task 2:** From Requirement to File-Processing Design: Biometric Attendance Files (8 Marks | K3 Apply)
3. **Task 3:** Design the Error Handling: Exceptions in the Attendance Processor (9 Marks | K4 Analyse)
4. **Task 4:** File and Exception Forensics (7 Marks | K4 Analyse)
5. **Task 5:** The Same Utility Organised Four Ways (7 Marks | K5 Evaluate)
6. **Task 6:** Design Challenge: College Fest Registration Desk (13 Marks | K5 Evaluate)

Task 1 — File-Processing Design Review: Diagnose and Repair (6 Marks | K4)

Problem Context: Hostel office script tallying mess bills from a raw text file.

1. Task 1.1: Diagnosis of 7 Design Deficiencies

Deficiency Identified	Technical Flaw & Risk in Hostel Office
1. Hard-coded File Path	Path `C:\\Users\\Office\\Desktop\\mess_oct.txt` breaks on Linux/Mac servers or different PC user profiles.
2. Unclosed File Pointer	`open()` without `close()` or `with` context manager risks file lock and memory leak.
3. In-Memory Slicing (`f.read()`)	Loads entire file at once into RAM; causes MemoryError crash when handling 50,000+ student lines.
4. Bare `except: pass`	Silently swallows all errors (KeyboardInterrupt, SyntaxError, corrupted data), losing corrupted bill entries without logging.
5. Integer Casting (`int(parts[2])`)	Fails with `ValueError` on floating decimal bills (e.g., `1250.50`).
6. Quadratic String Concatenation	`report = report + ...` inside loop exhibits $O(N^2)$ character copying, severely degrading performance.
7. Unhandled Trailing Newline	Blank line at EOF causes `IndexError` on `parts[1]` and `parts[2]`.

2. Task 1.2: Repaired Python Script

```
def process_mess_bills(filename):
    total = 0.0
    valid_records = []
    rejected_lines = []

    with open(filename, "r", encoding="utf-8") as f:
        for line_num, line in enumerate(f, start=1):
            clean_line = line.strip()
            if not clean_line:
                continue  # Skip trailing blank lines
            parts = clean_line.split(",")
            if len(parts) != 3:
                rejected_lines.append((line_num, line, "Incorrect column count"))
                continue
            try:
                reg_no, name, amount_str = parts[0].strip(), parts[1].strip(), parts[2].strip()
                amount = float(amount_str)
                total += amount
                valid_records.append(f"{name} {amount:.2f}")
            except ValueError:
                rejected_lines.append((line_num, line, "Invalid decimal amount"))

    report = "\n".join(valid_records)
    return report, total, rejected_lines
```

3. Task 1.3 & Alternative Complexity Analysis

Design Defense: The repaired script streams lines lazily (O(1) memory bound), automatically manages OS file handles via `with`, captures exact line failures in `rejected_lines` without crashing, and formats string reports in O(N) using `join()`.

Complexity Comparison: Reading full file via `f.read()` requires O(File Size) peak memory. Line-by-line iteration consumes O(1) buffer space (single line length).

Task 2 — From Requirement to File-Processing Design (8 Marks | K3)

Problem Context: Biometric attendance system producing daily classroom text files.

1. Task 2.1 & 2.2: Data Modeling and File Operations

- **Input File Format:** `<RegisterNumber>,<YYYY-MM-DD>,<P|A>` (e.g., `192311001,2026-10-08,P`).
- **Data Structures:** Accumulator Dictionary `student_counts = {reg_no: {"P": count, "A": count}}`.
- **File Modes & Encoding:** Reading daily files with mode `"r"` and `encoding="utf-8"`. Writing monthly reports with mode `"w"` and log files with mode `"a"`.

2. Task 2.3: Pseudocode for Monthly Processing

```
FUNCTION ProcessMonthlyAttendance(folder_path): student_counts = Dictionary() damaged_log = List() FOR E
```

3. Task 2.4: Execution Trace

Trace Example: Input File `day01.txt` with lines: `192311001,2026-10-08,P` and damaged line `192311002,2026-10-08,X`.

- **Line 1:** Parsed correctly $\rightarrow$ `student_counts['192311001']['P']` incremented from 0 to 1.
- **Line 2:** Status `'X'` invalid $\rightarrow$ Flagged and logged: `("day01.txt", Line 2, "192311002,2026-10-08,X")`. Execution continues smoothly without aborting.

Task 3 — Design Error Handling: Exceptions (9 Marks | K4)

1. Task 3.1: Failure Mode Mapping

Failure Mode	Built-in Exception	Mitigation Action
Missing input folder/file	`FileNotFoundError`	Notify user and prompt for valid path directory.
Restricted file access	`PermissionError`	Skip unreadable file and log administrative alert.
Non-UTF8 byte sequence	`UnicodeDecodeError`	Re-open with fallback encoding or log file corruption.
Missing comma delimiter	`InvalidRecord` (Custom)	Log line number and skip line processing.

2. Complete the Code Solution (TODO 1 & TODO 2)

```
import os, tempfile

class InvalidRecord(Exception):
    """Raised when a line of the attendance file is not in the expected format."""
    pass

def parse_line(line):
    parts = line.strip().split(",")
    if len(parts) != 3:
        raise InvalidRecord(f"expected 3 fields, found {len(parts)}")
    reg, date, status = parts[0].strip(), parts[1].strip(), parts[2].strip()
    if status not in ("P", "A"):
        raise InvalidRecord(f"invalid status '{status}'")
    return reg, date, status

def summarise(path):
    present, absent = 0, 0
    errors = []
    try:
        with open(path, "r", encoding="utf-8") as f:
            for line_num, line in enumerate(f, start=1):
                if not line.strip():
                    continue
                try:
                    _, _, status = parse_line(line)
                    if status == "P":
                        present += 1
                    else:
                        absent += 1
                except InvalidRecord as e:
                    errors.append((line_num, str(e)))
    except FileNotFoundError:
        return 0, 0, [(-1, "File not found")]
    return present, absent, errors

# Test Execution matching Expected Output
data = ("192311001,2026-10-08,P\n192311002,2026-10-08,A\n"
        "192311003,2026-10-08\n192311004,2026-10-08,L\n")
with tempfile.NamedTemporaryFile("w", suffix=".csv", delete=False, encoding="utf-8") as tmp:
    tmp.write(data)
    tmp_path = tmp.name

print(summarise(tmp_path))
os.remove(tmp_path)
# Output: (1, 1, [(3, "expected 3 fields, found 2"), (4, "invalid status 'L'")])
```

Task 4 — File and Exception Forensics (7 Marks | K4 - Analyse)

Problem Context: Library book return logger failing to preserve logs and stranding open handles.

1. Task 4.1: Locating Defects in Buggy Code

- **Defect 1 (Data Overwrite):** `open("returns.log", "w")` uses write mode, which truncates existing log contents every run. Only the final day's records persist.
- **Defect 2 (Missing Newline):** `log.write(b)` writes raw strings without trailing `\n`, joining distinct book titles together.
- **Defect 3 (Resource Leak):** `log.close()` occurs after `log.write(b)`. If `write()` raises an exception during execution, `log.close()` is skipped, leaving the file descriptor locked in memory.

2. Task 4.3: Repaired Forensics Program

```
def save_returns_repaired(books, log_filename="returns.log"):
    # Fix: Use mode "a" for append, with statement for auto-closing, and add \n
    with open(log_filename, "a", encoding="utf-8") as log:
        for b in books:
            log.write(b.strip() + "\n")

def process_day_file_repaired(day_file):
    try:
        with open(day_file, "r", encoding="utf-8") as f:
            books = [line.strip() for line in f if line.strip()]
        save_returns_repaired(books)
    except FileNotFoundError:
        with open("error.log", "a", encoding="utf-8") as err:
            err.write(f"Missing file error: {day_file}\n")
        print(f"Missing file logged: {day_file}")
```

3. Task 4.4: Prevention Rules

- **Rule 1:** Always use context managers (`with open(...)`) to guarantee handle closure regardless of runtime exceptions.
- **Rule 2:** Use mode `"a"` when accumulating multi-session data logs to prevent accidental truncation.

Task 5 — The Same Utility Organised Four Ways (7 Marks | K5 - Evaluate)

Problem Context: Structuring attendance processing utilities across different Python architecture paradigms.

1. Four Code Architecture Paradigms

Organisation	Structure Description	Refactoring Maintenance Effort
(a) Single Script	All reader, stats, and report code in `attendance.py`.	High effort. Editing requirement requires searching monolith file.
(b) Single Module	`attendance_lib.py` imported by `main.py`.	Moderate effort. Logic separated from driver script.
(c) Three Modules	Flat folder containing `reader.py`, `stats.py`, `report.py`.	Low effort. Modify only `stats.py` and `report.py`.
(d) Package Structure	Package `attendance/` with `__init__.py` & submodules.	Minimal effort. Clean encapsulated namespace boundaries.

2. Module Import Syntax Comparison

- `import attendance.stats`: Imports full path. Calls require qualified names (e.g., `attendance.stats.compute()`). Avoids namespace pollution.
- `from attendance import stats`: Imports module reference into current scope. Calls use `stats.compute()`. Recommended balance.
- `from attendance.stats import percentage`: Imports function directly. Calls use `percentage()`. Highest convenience, potential naming conflicts.

Task 6 — Design Challenge: College Fest Registration Desk (13 Marks | K5)

Problem Context: Merging, validating, and logging college fest registrations from 5 department files.

1. Stage 1 & 2: Problem Interpretation & Flow Design

```
+----------------------------+ | 5 Department Input Files | +-------------+-------------+ | v +------
```

2. Stage 3 & 4: Package Architecture & Pseudocode

```
# Package Layout: fest_registration/
# ├── __init__.py
# ├── reader.py
# ├── validator.py
# └── writer.py

# Pseudocode: validator.py
FUNCTION ValidateEmail(email):
    RETURN "@" IN email AND "." IN email.split("@")[-1]

FUNCTION ProcessDepartmentFiles(file_list):
    seen_regs = Set()
    clean_records = List()
    errors = List()

    FOR EACH path IN file_list:
        IF NOT Path(path).exists():
            errors.append(f"Missing File: {path}")
            CONTINUE
        WITH OPEN path AS f:
            FOR line_num, line IN enumerate(f):
                parts = line.strip().split(";")
                IF len(parts) < 5:
                    errors.append(f"{path}:{line_num} Invalid Columns")
                    CONTINUE
                reg, name, dept, email, events = parts[0], parts[1], parts[2], parts[3],
parts[4:]
                IF reg IN seen_regs:
                    CONTINUE  # Deduplicate silently
                IF NOT ValidateEmail(email):
                    errors.append(f"{path}:{line_num} Invalid Email: {email}")
                    CONTINUE
                seen_regs.add(reg)
                clean_records.append((reg, name, dept, email, events))
    RETURN clean_records, errors
```

3. Stage 5: Defense & Complexity Analysis

Deduplication Method	Time Complexity	Space Complexity	Scalability Evaluation
List Scanning (Searching earlier entries)	$O(N^2)$	O(N)	Extremely slow for 100,000 registrations (~10^{10} operations).
Set Lookup (`seen_regs.add()`)	O(N) Average	O(N)	Processes 100,000 registrations in sub-second time using hash lookup.